Asynchronous Result Pipelines in C#

Asynchronous Result Pipelines in C#: Task<Result> and LINQ-Based Monadic Composition for AI Applications

Version: v0.2.0
DOI: 10.5281/zenodo.20458492
Author: Takuya Sogawa
ORCID: 0009-0009-7499-2595

Abstract

This technical note defines a practical asynchronous result pipeline pattern for C# using `Task<Result<T>>` and LINQ query syntax. The pattern combines two different execution contexts: asynchronous computation represented by `Task<T>`, and predictable domain failure represented by `Result<T>`. In AI applications, failures such as schema mismatch, model refusal, token-limit overflow, unsafe output, tool rejection, or clarification requirements are often not exceptional system failures. They are ordinary states of the inference domain and should be represented as values.

The paper shows how C# query expressions can be used as a lightweight monadic composition mechanism by implementing `Select` and `SelectMany` for `Task<Result<T>>`. This enables declarative, fail-closed, short-circuiting pipelines without treating predictable failures as exceptions. The note positions the pattern as an implementation-level companion to Interface-Led Architecture (ILA): the pipeline separates Provider, Observer, and Operator responsibilities while preserving failure information as an explicit contract.

The contribution of this paper is to present `Task<Result<T>>` not merely as an idiom, but as a contract-oriented control-flow primitive for AI-era C# systems: asynchronous, typed, composable, AOT-friendly, and suitable for deterministic failure propagation in governed inference pipelines.

1. Introduction

Modern C# software routinely composes asynchronous operations. Network calls, file access, database queries, HTTP APIs, and model-provider calls naturally return `Task<T>` or related awaitable types. At the same time, many application failures are not exceptional. A user input may be invalid, an AI model may refuse a request, a response may not match a JSON schema, a policy gate may deny execution, or a downstream tool may require clarification.

Traditional exception handling is valuable for unexpected faults, programming errors, cancellation, infrastructure failures, and unrecoverable conditions. However, using exceptions as the primary representation for ordinary domain outcomes makes the control flow harder to reason about. It also hides failure in a side channel rather than exposing it in the method signature.

This paper proposes `Task<Result<T>>` as a practical C# abstraction for asynchronous workflows whose expected outcome may be success or typed failure. The pattern is especially useful in AI applications, where uncertainty, validation failure, clarification, refusal, and fallback are normal parts of the domain.

2. Context-Carrying Values and Monadic Composition

A monadic style treats a value together with its context. In this paper, the relevant contexts are:

- `Task<T>`: a value that will be produced asynchronously.
- `Result<T>`: a value that either succeeded or failed with structured error information.
- `Task<Result<T>>`: an asynchronous computation that may produce either a successful value or a predictable failure.

The key operation is `Bind`, represented in C# LINQ by `SelectMany`. `Bind` applies the next computation only when the previous result succeeded. If the previous computation failed, the pipeline short-circuits and propagates the failure.

This paper does not claim to provide a fully general monad-transformer framework for C#. C# lacks higher-kinded types, so the pattern is implemented through concrete extension methods for the specific composite type `Task<Result<T>>`.

3. Result Type Design

A minimal production-oriented `Result<T>` should carry a value on success and a structured error on failure. A string-only error is acceptable for demonstrations, but a dedicated error type is preferable in real systems.

```
public readonly record struct Error(
    string Code,
    string Message,
    string? Detail = null);

public readonly struct Result<T>
{
    private readonly T? _value;

    public bool IsSuccess { get; }
    public bool IsFailure => !IsSuccess;
    public T Value => IsSuccess
        ? _value!
        : throw new InvalidOperationException("No success value.");
    public Error? Error { get; }

    private Result(T value)
    {
        IsSuccess = true;
        _value = value;
        Error = null;
    }

    private Result(Error error)
    {
        IsSuccess = false;
        _value = default;
        Error = error;
    }

    public static Result<T> Success(T value) => new(value);
    public static Result<T> Failure(Error error) => new(error);
}
```

This representation intentionally keeps predictable failure in the value domain. Exceptions may still occur for programming errors or infrastructure faults, but they are no longer the main mechanism for expected validation and policy outcomes.

4. LINQ Support for `Task<Result<T>>`

C# query syntax is pattern-based. If a type exposes compatible `Select` and `SelectMany` methods, the compiler can translate query expressions into method calls. For `Task<Result<T>>`, these methods define asynchronous mapping and binding.

```
public static class TaskResultExtensions
{
    public static async Task<Result<U>> Select<T, U>(
        this Task<Result<T>> source,
        Func<T, U> selector)
    {
        var result = await source.ConfigureAwait(false);
        if (result.IsFailure)
        {
            return Result<U>.Failure(result.Error!.Value);
        }

        return Result<U>.Success(selector(result.Value));
    }

    public static async Task<Result<U>> SelectMany<T, C, U>(
        this Task<Result<T>> source,
        Func<T, Task<Result<C>>> collectionSelector,
        Func<T, C, U> resultSelector)
    {
        var first = await source.ConfigureAwait(false);
        if (first.IsFailure)
        {
            return Result<U>.Failure(first.Error!.Value);
        }

        var second = await collectionSelector(first.Value)
            .ConfigureAwait(false);
        if (second.IsFailure)
        {
            return Result<U>.Failure(second.Error!.Value);
        }

        return Result<U>.Success(
            resultSelector(first.Value, second.Value));
    }
}
```

With these extension methods, a governed asynchronous pipeline can be written as a declarative sequence.

```
Task<Result<ParsedData>> ExecuteAsync(string input)
{
    return
        from prompt in BuildPromptAsync(input)
        from output in CallModelAsync(prompt)
        from parsed in ParseOutputAsync(output)
        select parsed;
}
```

If `BuildPromptAsync`, `CallModelAsync`, or `ParseOutputAsync` returns a failure, the later steps are skipped automatically. The pipeline becomes fail-closed by construction.

5. AI Application Pattern

AI applications often process a sequence such as:

1. build a prompt or structured context;
2. call a model provider;
3. validate the output;
4. parse the response;
5. apply a policy gate;
6. execute or return the result.

Each step may fail in a predictable way. Examples include:

- missing required input;
- prompt construction failure;
- model refusal;
- invalid JSON;
- schema mismatch;
- policy denial;
- unsafe tool invocation;
- insufficient confidence;
- clarification required.

Representing these outcomes as `Result<T>` allows the pipeline to preserve domain semantics.

```
Task<Result<Answer>> GovernedAnswerAsync(UserInput input)
{
    return
        from context in AssembleContextAsync(input)
        from prompt in GeneratePromptAsync(context)
        from modelText in CallModelAsync(prompt)
        from parsed in ParseJsonAsync(modelText)
        from decision in EvaluatePolicyAsync(parsed)
        from answer in RenderAnswerAsync(decision)
        select answer;
}
```

This style makes the happy path readable while keeping failure propagation explicit and type-directed.

6. Relationship to Provider, Observer, and Operator

In ILA terms, the `Task<Result<T>>` pattern can be interpreted through the Provider-Observer-Operator abstraction discipline.

Role	Example in an AI pipeline	Failure semantics
Provider	model provider, VFS provider, context provider	unavailable, refused, not capable
Observer	schema validator, policy observer, risk observer	invalid, unsafe, ambiguous
Operator	parser, transformer, renderer, executor	transform failed, denied, incomplete

The monadic pipeline does not erase these roles. It provides a way to compose them while preserving the result contract at every boundary.

A Provider supplies values. An Observer evaluates state. An Operator transforms state. `Task<Result<T>>` allows all three to participate in a common asynchronous failure contract.

7. Exceptions and Result Values

This paper does not propose eliminating exceptions. Instead, it separates two categories:

Category	Recommended representation
predictable domain failure	`Result<T>`
validation failure	`Result<T>`
policy denial	`Result<T>`
clarification required	`Result<T>`
programming error	exception
infrastructure failure	exception or mapped failure
cancellation	`CancellationToken` / cancellation exception

The boundary between exception and result is architectural. A low-level HTTP timeout may begin as an exception, but an application service may map it into a `Result<T>` failure such as `provider.timeout` if it is an expected operational outcome.

The central rule is this:

> Expected domain outcomes should travel as values. Unexpected faults may travel as exceptions.

8. Comparison with Reactive Extensions

Reactive Extensions (`IObservable<T>`) model values over time. They are well suited for streams, UI events, sensor data, and event-driven workflows.

`Task<Result<T>>` models a single asynchronous computation with explicit success or failure. It is well suited for request-response workflows and sequential AI inference pipelines.

Aspect	`IObservable<T>`	`Task<Result<T>>`
Cardinality	zero to many values	one eventual result
Primary context	time and stream propagation	async completion and failure
Composition	stream operators	LINQ query / `SelectMany`
Error style	stream error channel	explicit result value
Best fit	event streams	governed request pipelines

Both approaches are useful. The point is not to replace reactive programming, but to select the abstraction whose context matches the problem.

9. Comparison with Rust and Haskell

Rust includes `Result<T, E>` as a standard error-handling type and uses the `?` operator for early return. The C# pattern in this paper is conceptually similar: failure propagates without nested conditional code.

Haskell's `Either` monad and `do` notation serve a similar role for sequential composition of computations that may fail. C# LINQ query syntax can be viewed as an analogous surface notation for types that implement the required query pattern.

The difference is that C# does not provide a general higher-kinded `Monad` interface. Instead, the pattern is encoded directly through concrete `Select` and `SelectMany` extension methods.

10. Modern .NET Considerations

The pattern is compatible with modern .NET design constraints:

- It uses ordinary C# types and extension methods.
- It does not require reflection or dynamic code generation.
- It is compatible with Native AOT when the surrounding code is also AOT-compatible.
- It can be implemented with `readonly struct` or `record struct` to reduce avoidable allocations.
- In hot paths, `ValueTask<Result<T>>` can be considered when profiling shows that it is beneficial.

`ValueTask` should not be used merely because it looks faster. It introduces additional usage constraints and should be chosen when there is a measured benefit, such as frequent synchronous completion.

11. Non-Claims and Limitations

This paper makes the following non-claims.

1. It does not claim that exceptions are obsolete.
2. It does not claim that `Task<Result<T>>` is a universal replacement for all error handling.
3. It does not provide a general monad-transformer framework for C#.
4. It does not prove monad laws for every possible `Result<T>` implementation.
5. It does not claim that LINQ query syntax is always preferable to explicit method calls.
6. It does not claim that all AI failures are predictable domain failures.

The intended claim is narrower: for asynchronous AI workflows with predictable validation, policy, parsing, and provider failures, `Task<Result<T>>` provides a clear and type-directed composition model.

12. Conclusion

`Task<Result<T>>` is a natural abstraction for C# AI applications that need to combine asynchronous execution with deterministic failure propagation. It turns predictable failures into values, keeps error semantics visible in the type signature, and enables LINQ-based declarative pipelines through `Select` and `SelectMany`.

In the AI era, uncertainty is not always an exception. Often it is the domain itself. A well-designed result pipeline allows that uncertainty to be governed, composed, validated, and safely short-circuited.

From the perspective of Interface-Led Architecture, this pattern turns asynchronous inference into a contract-preserving pipeline: Providers supply values, Observers validate state, Operators transform outputs, and failures remain explicit at every boundary.

References

1. Microsoft. “Task asynchronous programming model.” Microsoft Learn. Available at: https://learn.microsoft.com/en-us/dotnet/csharp/asynchronous-programming/task-asynchronous-programming-model.
2. Microsoft. “Projection operations in LINQ.” Microsoft Learn. Available at: https://learn.microsoft.com/en-us/dotnet/csharp/linq/standard-query-operators/projection-operations.
3. Microsoft. “C# language specification: Query expressions and query expression translation.” Microsoft Learn. Available at: https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/language-specification/expressions.
4. Microsoft. “Best practices for exceptions.” Microsoft Learn. Available at: https://learn.microsoft.com/en-us/dotnet/standard/exceptions/best-practices-for-exceptions.
5. Microsoft. “Native AOT deployment overview.” Microsoft Learn. Available at: https://learn.microsoft.com/en-us/dotnet/core/deploying/native-aot/.
6. Microsoft. “ValueTask Struct.” Microsoft Learn. Available at: https://learn.microsoft.com/en-us/dotnet/api/system.threading.tasks.valuetask-1.

7. The Rust Project Developers. “Recoverable Errors with Result.” *The Rust Programming Language*. Available at: https://doc.rust-lang.org/book/ch09-02-recoverable-errors-with-result.html.
8. Wadler, Philip. “Monads for Functional Programming.” In *Advanced Functional Programming*, Lecture Notes in Computer Science, vol. 925, Springer, 1995, pp. 24-52. DOI: 10.1007/3-540-59451-5_2.
9. ReactiveX. “ReactiveX: An API for asynchronous programming with observable streams.” Available at: https://reactivex.io/.
10. Sogawa, Takuya. “Interface-Led Architecture (ILA): A Software Development Methodology for the AI Era, Validated by the AIKernel Execution Model.” Zenodo, 2026. DOI: 10.5281/zenodo.20290614.
11. Sogawa, Takuya. “Provider-Observer-Operator: A Role-Based Abstraction Discipline for Interface-Led Architecture.” Zenodo, 2026. DOI: 10.5281/zenodo.20322690.
12. Sogawa, Takuya. “Prompt-State Machines: Applying Interface-Led Architecture to Structure LLM Reasoning.” Zenodo, 2026. DOI: 10.5281/zenodo.20323512.